



RELAZIONE MINI-ALU

A CURA DI:

MARIAIDA PERRI – MAT.239943

ANDREEA ROXANA MANOLACHE – MAT.245906

ANDREA MAZZA – MAT.240059

FRANCESCO PIO RUFFO – MAT.240044

A.A. 2024/25

ARCHITETTURA DI RIFERIMENTO E SCELTE PROGETTUALI

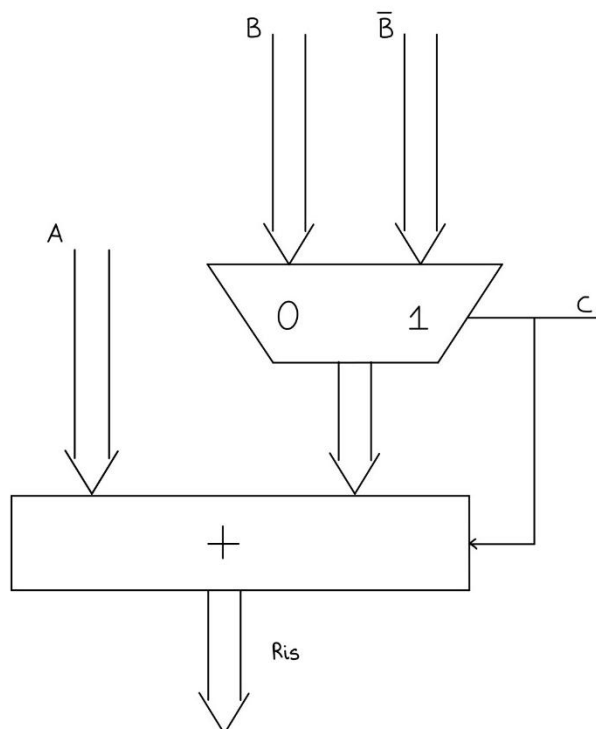
L'obiettivo del progetto è la realizzazione di un '**mini-ALU**', ovvero un circuito che conserva la logica di un Arithmetic Logic Unit tradizionale, presentato in una versione semplificata.

Il mini-ALU infatti, a differenza dell'ALU tradizionale che consente di effettuare svariate operazioni (moltiplicazione, divisione, operazioni logiche, operazioni di SHIFT e rotazione), supporta le seguenti operazioni:

C	uscita
0	A+B
1	A-B

Come emerge dalla tabella sopra riportata, il componente descritto riceve come ingressi due operandi ad $n - bit$ (**A**, **B**) ed un bit di controllo (**C**).

Le operazioni effettuate dipendono quindi dal bit di controllo; in particolare, nel caso in cui $C = '0'$ l'output prodotto sarà A+B, altrimenti sarà A-B.



CARATTERISTICHE GENERALI DEL CODICE

❖ TIPO STD_LOGIC

Per la realizzazione del circuito si è fatto utilizzo del tipo 'std_logic'.

Esso fa parte del pacchetto 'std_logic_1164' della libreria 'IEEE' e rappresenta un segnale che può avere più stati oltre a quelli definiti dal tipo Bit (0 e 1). In particolare, può rappresentare nove diversi stati:

U	Uninitialized
X	Unknown
0	Force 0
1	Force 1
Z	High Impedance
W	Weak Unknown
L	Weak Low
H	Weak High
-	Don't Care

Il tipo 'std_logic' può essere utilizzato anche in array, ad esempio, per rappresentare bus o vettori di segnali logici. Per gli array esiste il tipo 'std_logic_vector', che è analogo a 'bit_vector', ma con i 9 stati possibili per ogni digit.

Questo tipo consente di simulare comportamenti realistici nei circuiti, altrimenti non rappresentabili con il solo tipo bit.

❖ GENERIC

Al fine di rendere il codice VHDL più flessibile e riutilizzabile, si è fatto uso del costrutto 'generic'.

Quest'ultimo permette di scrivere un codice in modo parametrico: mediante un parametro, o una serie di parametri, che vengono stabiliti al momento della sintesi o della simulazione, si può adattare il codice al circuito che si vuole realizzare. Ad esempio, il 'generic' permette di variare la dimensione degli operandi di un circuito senza intervenire direttamente sul codice.

Nell'implementazione richiesta, la caratterizzazione del circuito viene effettuata a **4,8,16 bit**.

CODICE VHDL MINI-ALU

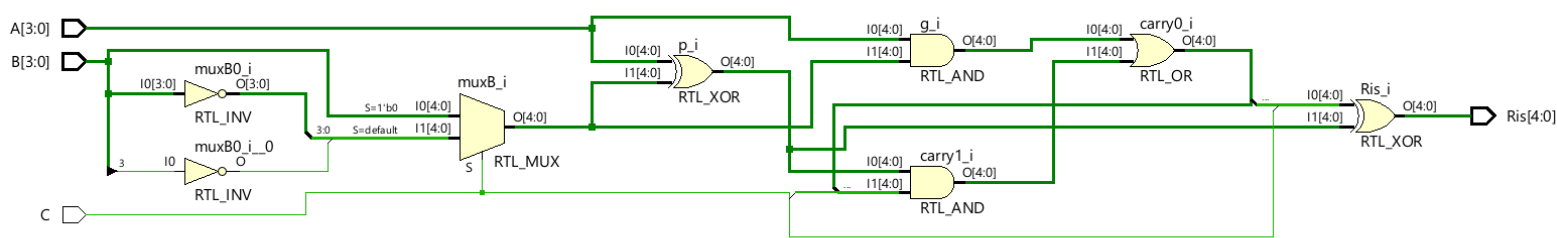
```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3
4  entity mini_alu is
5      generic(n:integer:=4);
6
7      port ( A,B : in std_logic_vector (n-1 downto 0);
8            C : in std_logic;
9            Ris : out std_logic_vector (n downto 0));
10 end mini_alu;
11
12 architecture my_alu of mini_alu is
13
14     signal p,g: std_logic_vector(n downto 0);
15     signal carry: std_logic_vector(n+1 downto 0);
16     signal muxB: std_logic_vector(n downto 0);
17     signal extA: std_logic_vector(n downto 0);
18
19 begin
20     extA <= A(n-1)&A;
21     muxB <= B(n-1)&B when C='0'
22         else (not B(n-1))&not B;
23     p <= extA xor muxB;
24     g <= extA and muxB;
25     carry(0) <= c;
26     carry(n+1 downto 1) <= g or(p and carry(n downto 0));
27     Ris <= carry(n downto 0) xor p;
28
29 end my_alu;
```

Utilizzando il costrutto ‘generic’, nella entity del circuito è necessario specificare un parametro di default; in questo modo, se nella caratterizzazione del circuito non dovesse essere specificato un nuovo parametro, verrà preso automaticamente quello presente nella entity. In questo caso, il parametro di default è fissato a 4.

La somma tra i due operandi viene eseguita in modo tradizionale, servendosi dei segnali propagate e generate. Per quanto riguarda la differenza, essa viene calcolata come $A + \text{not } B + 1$, sfruttando così la proprietà del complemento a due. La differenza in complemento a due si può trasformare infatti in una somma, aggiungendo il complemento a due del numero da sottrarre. A questo scopo, si è fatto utilizzo del costrutto ‘when-else’ grazie al quale viene preso il valore di B corretto, a seconda del valore di C.

Nel codice si è prevista l’estensione di un bit dei due operandi in modo da garantire che l’operazione di somma o differenza venga eseguita correttamente anche nel caso in cui il risultato esca dai limiti della rappresentazione originale degli operandi.

SCHEMATIC



TESTBENCH

Per poter effettuare una simulazione esaustiva del circuito sarebbe necessario considerare tutte le possibili combinazioni degli ingressi. Tuttavia, il numero totale di combinazioni rende il tempo di simulazione considerevolmente elevato per circuiti con un elevato numero di ingressi (come nella caratterizzazione a 16 bit).

Per questo motivo, si è proceduto col selezionare un sottoinsieme rappresentativo delle varie combinazioni possibili; ciò consente comunque di poter verificare il comportamento del circuito posto sotto test.

In un contesto reale dovremmo effettuare la simulazione tenendo in considerazione questi range di valori:

Configurazione a 4 bit

```
1 for ia in -23 to 23-1 loop
2   for ib in -23 to 23-1 loop
```

Configurazione a 8 bit

```
1 for ia in -27 to 27-1 loop
2   for ib in -27 to 27-1 loop
```

Configurazione a 16 bit

```
1 for ia in -215 to 215-1 loop
2   for ib in -215 to 215-1 loop
```

Durante la fase implementativa, inoltre, è stato previsto l'inserimento del segnale *risAtteso*, il cui valore è calcolato utilizzando la libreria *IEEE.STD_LOGIC_Arith*. Questo segnale rappresenta l'output atteso per ogni combinazione di input, consentendo quindi di verificare la corrispondenza tra il risultato effettivo e quello atteso; ciò garantisce un check 'automatico' in merito al corretto funzionamento del circuito.

Caratterizzazione a 4 bit

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_Arith.ALL;
4
5 entity sim_mini_alu is
6     generic(n: integer:=4);
7 end sim_mini_alu;
8
9 architecture my_sim of sim_mini_alu is
10     signal A,B: std_logic_vector(n-1 downto 0);
11     signal CC: std_logic_vector(0 downto 0);
12     signal C: std_logic;
13     signal Ris: std_logic_vector(n downto 0);
14     signal risAtteso: integer;
15
16     component mini_alu
17     port( A,B : in std_logic_vector (n-1 downto 0);
18          C : in std_logic;
19          Ris : out std_logic_vector (n downto 0));
20     end component;
21 begin
22     CUT: mini_alu port map(A,B,CC(0),Ris);
23     C<=CC(0);
24
25     process begin
26
27     for ic in 0 to 1 loop
28         CC<=conv_std_logic_vector(ic,1);
29
30     for ia in -8 to 8 loop
31         A <= conv_std_logic_vector(ia,4);
32         for ib in -4 to 5 loop
33             B <= conv_std_logic_vector(ib,4);
34             if ic=0 then
35                 risAtteso <= ia+ib;
36             else risAtteso <= ia-ib;
37             end if;
38             wait for 20 ns;
39             end loop;
40         end loop;
41     end loop;
42     wait for 10 ns;
43
44 end process;
45
46 end my_sim;
```

Caratterizzazione a 8 bit

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_Arith.ALL;
4
5  entity sim_mini_alu_8 is
6      generic(n: integer:=8);
7  end sim_mini_alu_8;
8
9  architecture my_sim_8 of sim_mini_alu_8 is
10     signal A,B: std_logic_vector(n-1 downto 0);
11     signal CC: std_logic_vector(0 downto 0);
12     signal C: std_logic;
13     signal Ris: std_logic_vector(n downto 0);
14     signal risAtteso: integer;
15
16     component mini_alu
17     port( A,B : in std_logic_vector (n-1 downto 0);
18          C : in std_logic;
19          Ris : out std_logic_vector (n downto 0));
20     end component;
21 begin
22     CUT: mini_alu port map(A,B,CC(0),Ris);
23     C<=CC(0);
24
25     process begin
26
27     for ic in 0 to 1 loop
28         CC<=conv_std_logic_vector(ic,1);
29
30     for ia in -32 to 32 loop
31         A <= conv_std_logic_vector(ia,8);
32         for ib in -16 to 28 loop
33             B <= conv_std_logic_vector(ib,8);
34             if ic=0 then
35                 risAtteso <= ia+ib;
36             else risAtteso <= ia-ib;
37             end if;
38             wait for 20 ns;
39             end loop;
40         end loop;
41     end loop;
42     wait for 10 ns;
43
44 end process;
45 end my_sim_8;
```

Come si può notare, la entity non è più vuota; bisogna specificare quale valore deve assumere il parametro 'generic'. Lo stesso parametro va specificato anche all'interno dell'architettura. Nel caso precedente ciò non è stato necessario in quanto il parametro di default del component è fissato a 4.

Caratterizzazione a 16 bit

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_Arith.ALL;
4
5  entity sim_mini_alu_16 is
6      generic(n: integer:=16);
7  end sim_mini_alu_16;
8
9  architecture my_sim_16 of sim_mini_alu_16 is
10     signal A,B: std_logic_vector(n-1 downto 0);
11     signal CC: std_logic_vector(0 downto 0);
12     signal C: std_logic;
13     signal Ris: std_logic_vector(n downto 0);
14     signal risAtteso: integer;
15
16     component mini_alu
17     port( A,B : in std_logic_vector (n-1 downto 0);
18          C : in std_logic;
19          Ris : out std_logic_vector (n downto 0));
20     end component;
21 begin
22     CUT: mini_alu port map(A,B,CC(0),Ris);
23     C<=CC(0);
24
25     process begin
26
27     for ic in 0 to 1 loop
28         CC<=conv_std_logic_vector(ic,1);
29
30     for ia in -40 to 18 loop
31         A <= conv_std_logic_vector(ia,16);
32         for ib in -27 to 15 loop
33             B <= conv_std_logic_vector(ib,16);
34             if ic=0 then
35                 risAtteso <= ia+ib;
36             else risAtteso <= ia-ib;
37             end if;
38             wait for 20 ns;
39         end loop;
40     end loop;
41     end loop;
42     wait for 10 ns;
43
44 end process;
45
46 end my_sim_16;
```

SIMULAZIONE BEHAVIORAL

Tenendo in considerazione i precedenti codici VHDL, è stata effettuata innanzitutto una simulazione behavioral, in modo da verificare il corretto funzionamento del circuito per gli ingressi specificati.

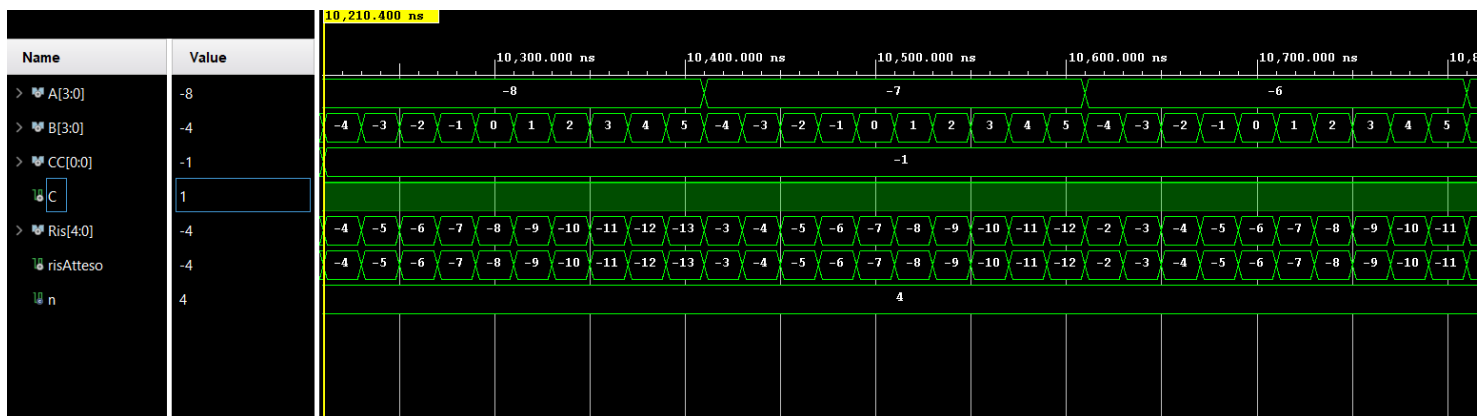
I risultati ottenuti, coincidenti con il valore del segnale *risAtteso*, sono stati i seguenti:

- **Caratterizzazione a 4 bit**

$C = '0' \rightarrow \text{somma}$

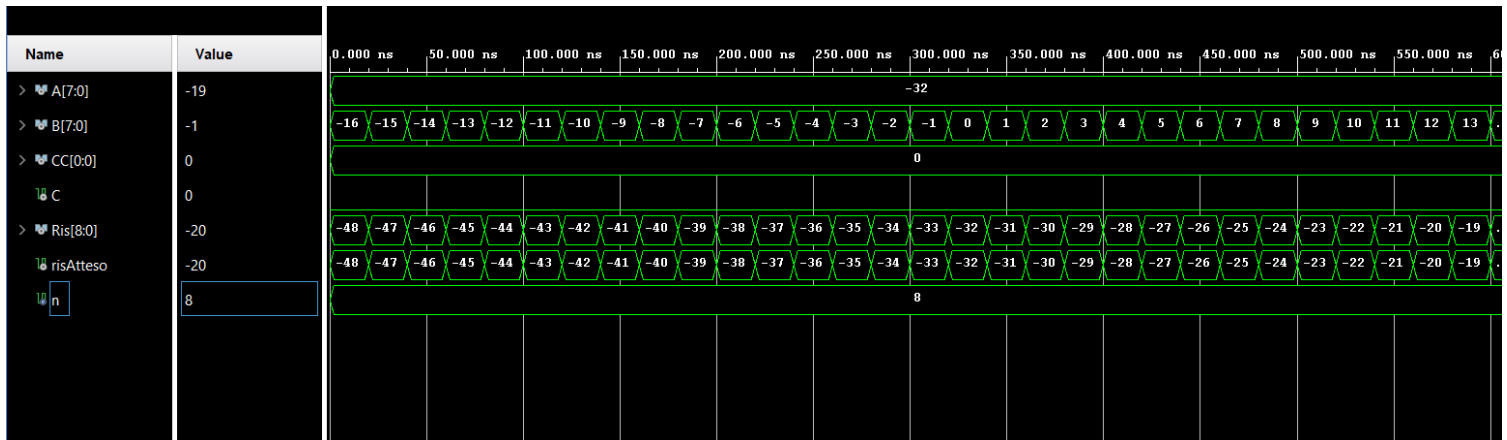


$C = '1' \rightarrow \text{differenza}$

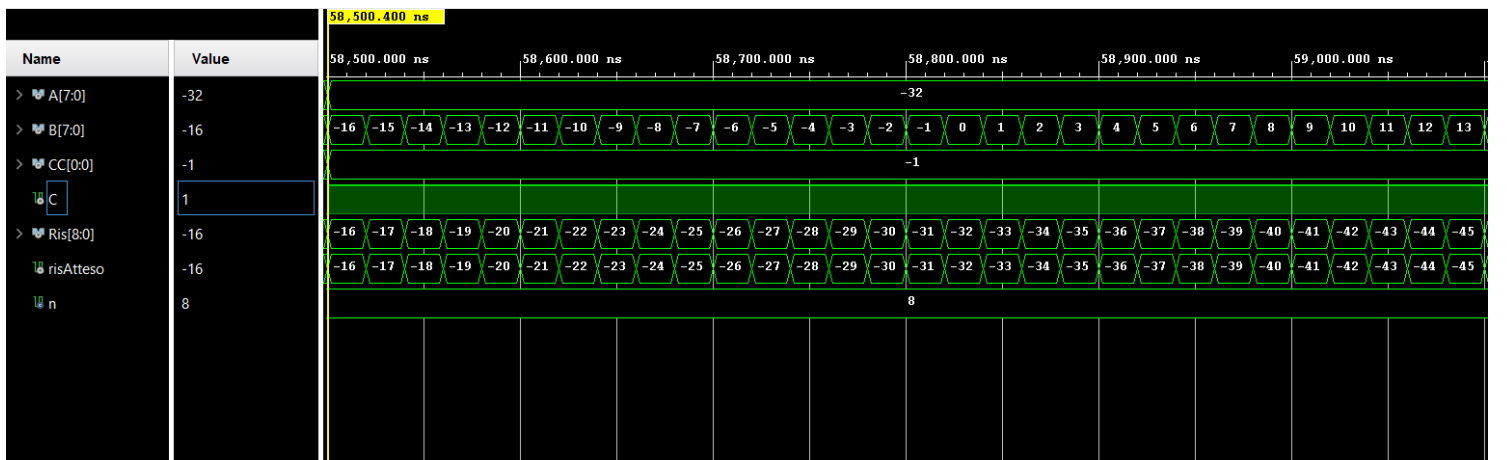


- Caratterizzazione a 8 bit

$C = '0' \rightarrow \text{somma}$

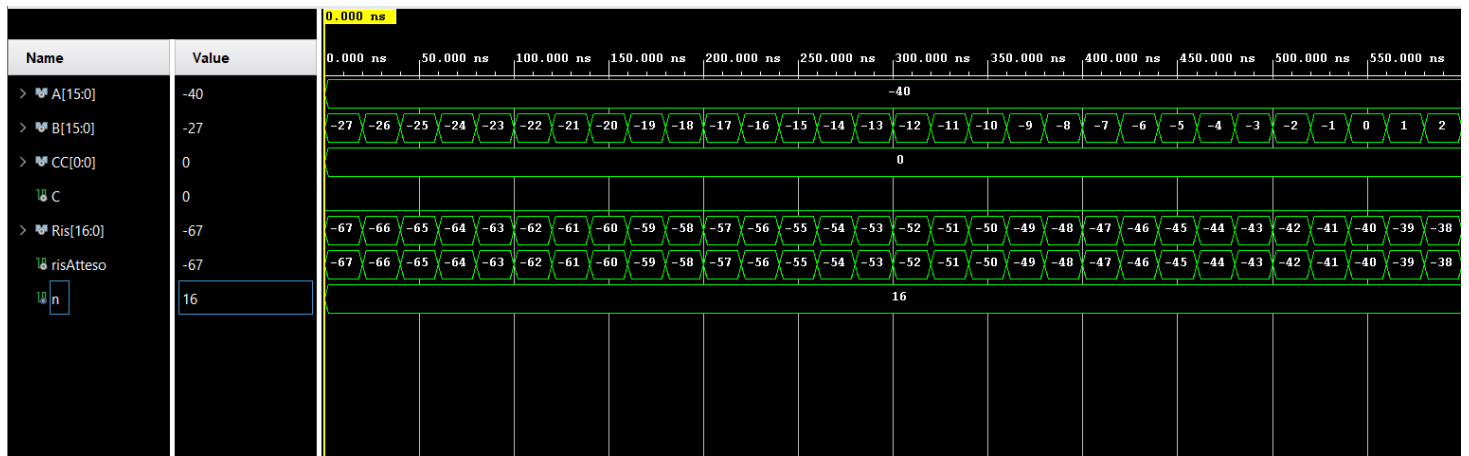


$C = '1' \rightarrow \text{differenza}$

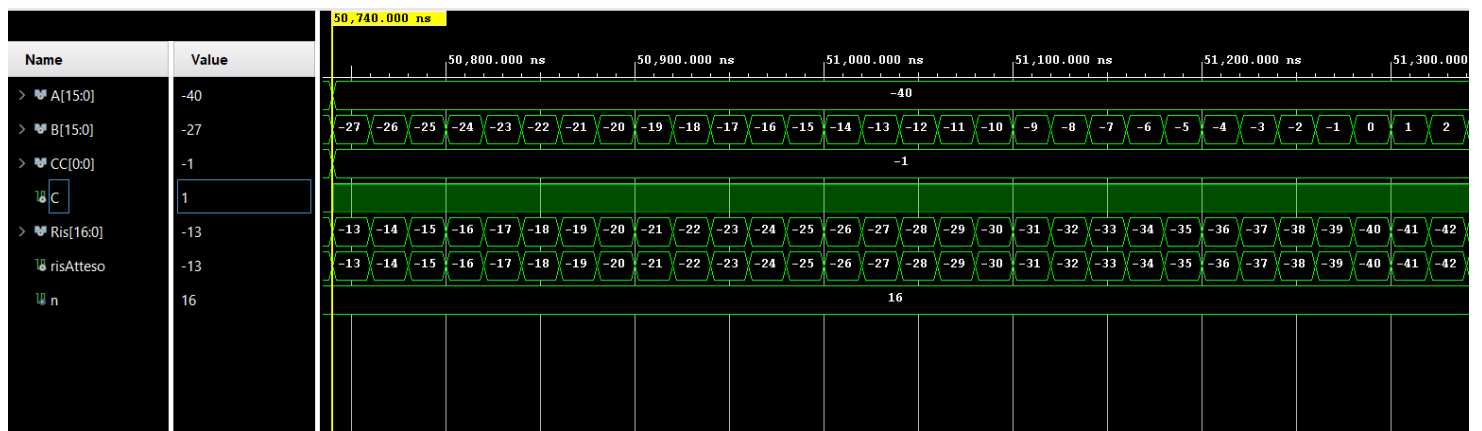


- Caratterizzazione a 16 bit

$C = '0' \rightarrow \text{somma}$



$C = '1' \rightarrow \text{differenza}$



SINTESI

- **Caratterizzazione a 4 bit**

Report sull'utilizzazione delle risorse (LUT)

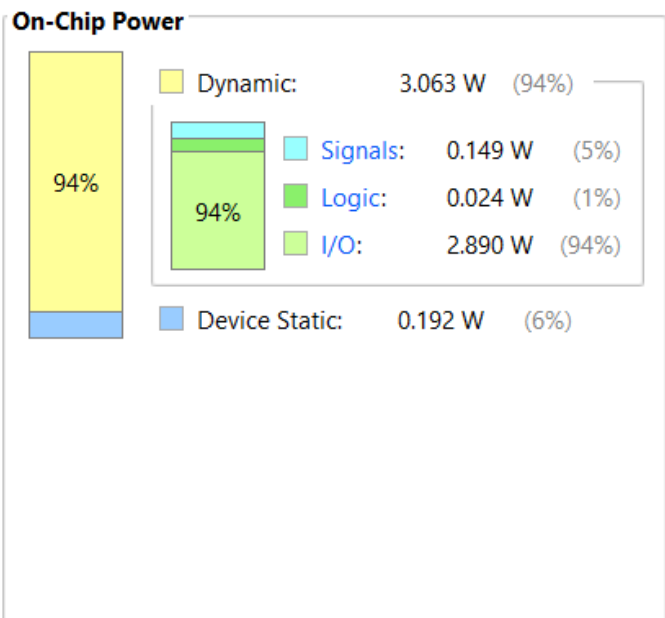
Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs	5	0	0	53200	<0.01
LUT as Logic	5	0	0	53200	<0.01
LUT as Memory	0	0	0	17400	0.00
Slice Registers	0	0	0	106400	0.00
Register as Flip Flop	0	0	0	106400	0.00
Register as Latch	0	0	0	106400	0.00
F7 Muxes	0	0	0	26600	0.00
F8 Muxes	0	0	0	13300	0.00

Dissipazione di potenza

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: **3.254 W**
Design Power Budget: **Not Specified**
Power Budget Margin: **N/A**
Junction Temperature: **62,5°C**
Thermal Margin: 37,5°C (3,0 W)
Effective θ_{JA} : 11,5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity



- **Caratterizzazione a 8 bit**

Report sull'utilizzazione delle risorse (LUT)

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	11	0	0	53200	0.02
LUT as Logic	11	0	0	53200	0.02
LUT as Memory	0	0	0	17400	0.00
Slice Registers	0	0	0	106400	0.00
Register as Flip Flop	0	0	0	106400	0.00
Register as Latch	0	0	0	106400	0.00
F7 Muxes	0	0	0	26600	0.00
F8 Muxes	0	0	0	13300	0.00

Dissipazione di potenza

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: **6.694 W (Junction temp exceeded!)**

Design Power Budget: **Not Specified**

Power Budget Margin: **N/A**

Junction Temperature: **102,2°C**

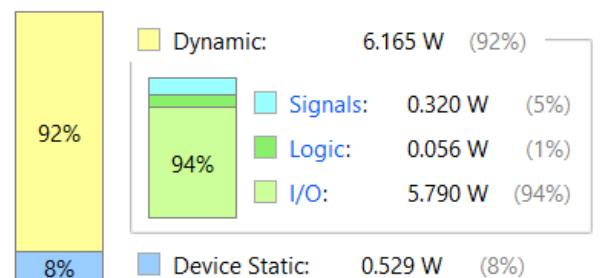
Thermal Margin: **-2,2°C (-0,1 W)**

Effective θ_{JA} : 11,5°C/W

Power supplied to off-chip devices: 0 W

Confidence level: **Low**

On-Chip Power



- **Caratterizzazione a 16 bit**

Report sull'utilizzazione delle risorse (LUT)

Site Type	Used	Fixed	Prohibited	Available	Util%
Slice LUTs*	32	0	0	53200	0.06
LUT as Logic	32	0	0	53200	0.06
LUT as Memory	0	0	0	17400	0.00
Slice Registers	0	0	0	106400	0.00
Register as Flip Flop	0	0	0	106400	0.00
Register as Latch	0	0	0	106400	0.00
F7 Muxes	0	0	0	26600	0.00
F8 Muxes	0	0	0	13300	0.00

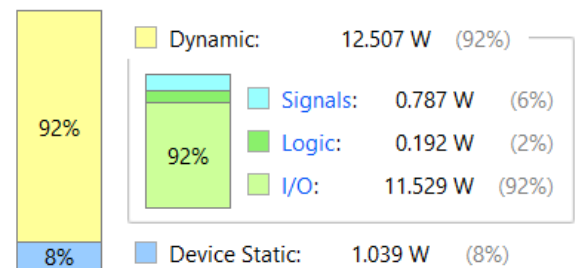
Dissipazione di potenza

Power estimation from Synthesized netlist. Activity derived from constraints files, simulation files or vectorless analysis. Note: these early estimates can change after implementation.

Total On-Chip Power: **13.547 W (Junction temp exceeded!)**
Design Power Budget: **Not Specified**
Power Budget Margin: **N/A**
Junction Temperature: **125,0°C**
Thermal Margin: **-81,2°C (-6,5 W)**
Effective θ_{JA} : 11,5°C/W
Power supplied to off-chip devices: 0 W
Confidence level: **Low**

[Launch Power Constraint Advisor](#) to find and fix invalid switching activity

On-Chip Power



Il codice VHDL viene mappato su una matrice di LUT.

Una LUT, acronimo di Look Up Table, è una rete di multiplexer in cui i segnali di ingresso e selezione vengono stabiliti in fase di sintesi. È possibile visualizzare il numero di LUT utilizzate grazie ai Report.

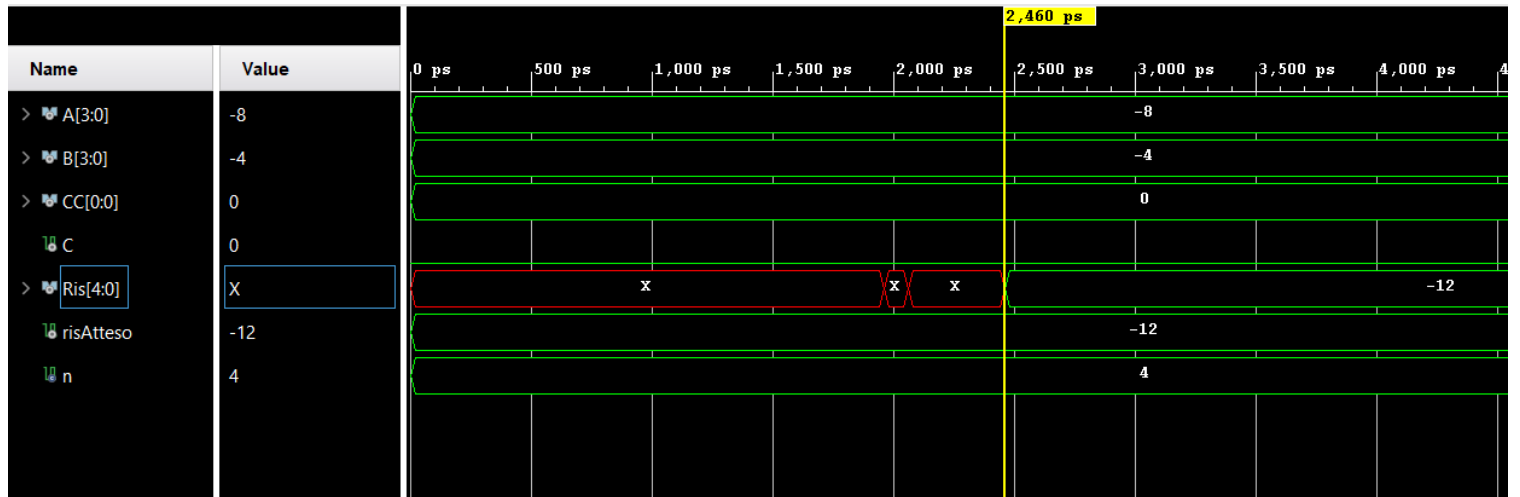
Di seguito vengono riassunti i risultati ottenuti nei tre casi:

N° bit	N° LUT
4	5
8	11
16	32

SIMULAZIONE POST-SINTESI

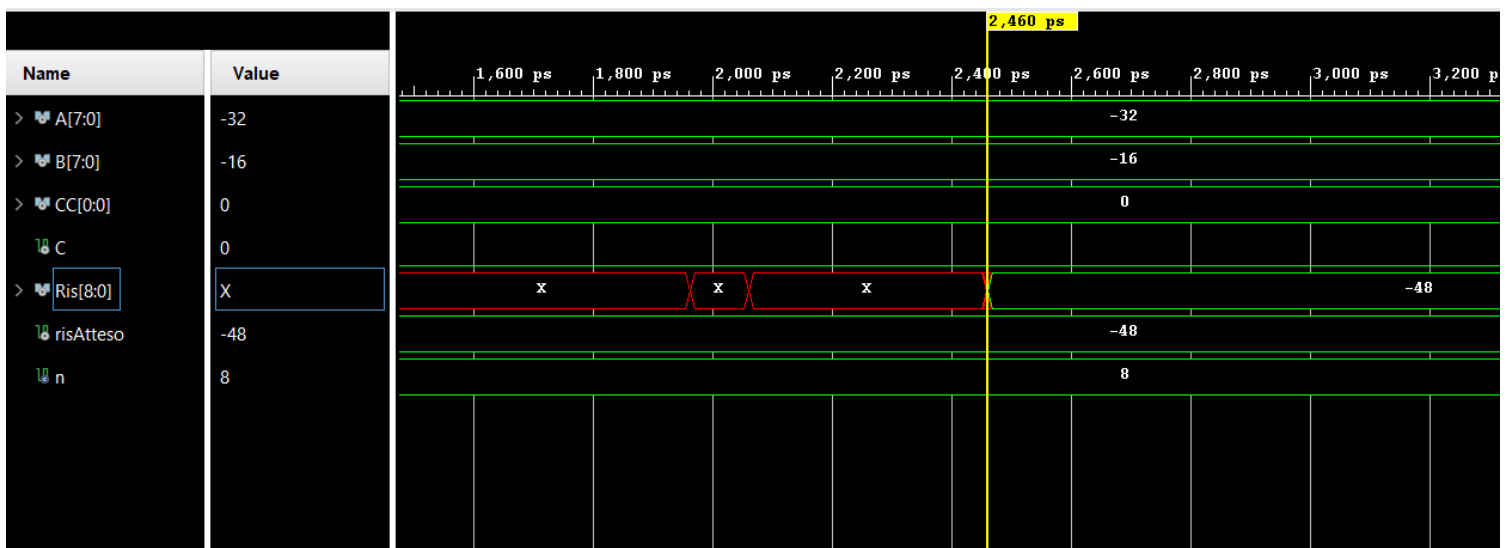
Mediante la simulazione post-sintesi, è possibile visualizzare i ritardi del circuito associati alle porte logiche. È presente in una fase iniziale lo stato X (“Unknown”), dovuto proprio ad una non specificazione causata dall’elaborazione degli ingressi in corso.

- **Caratterizzazione a 4 bit**

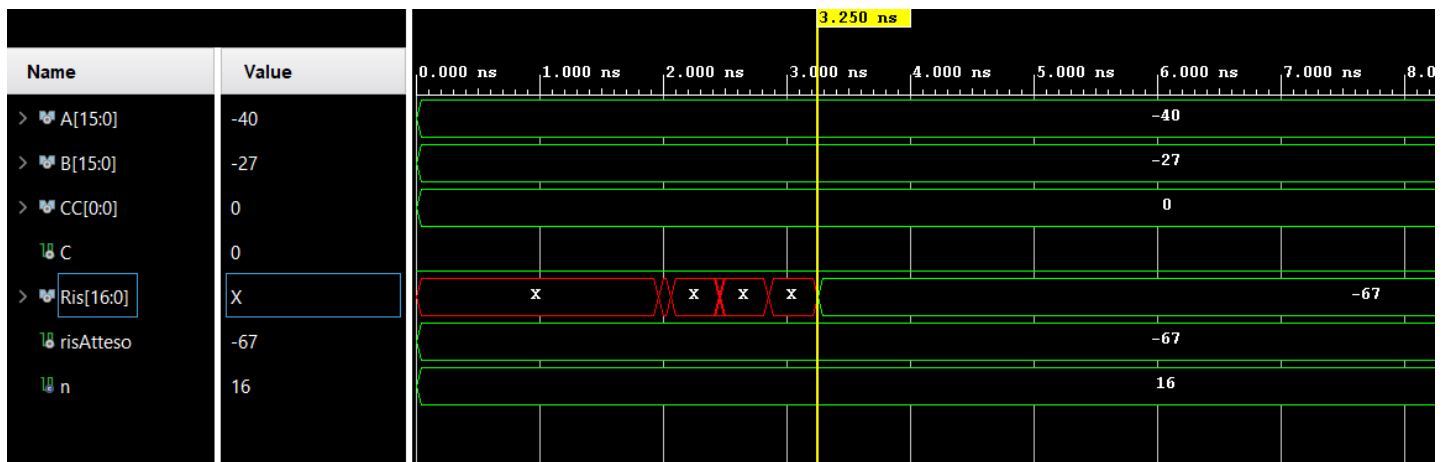


Come mostra la precedente immagine, è presente un ritardo di 2,460 ps.

- **Caratterizzazione a 8 bit**



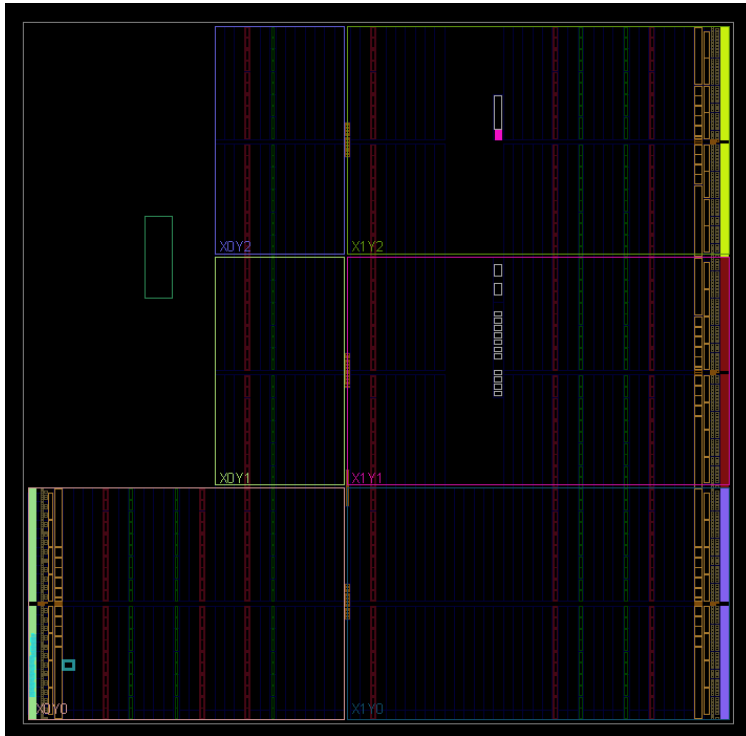
- Caratterizzazione a 16 bit



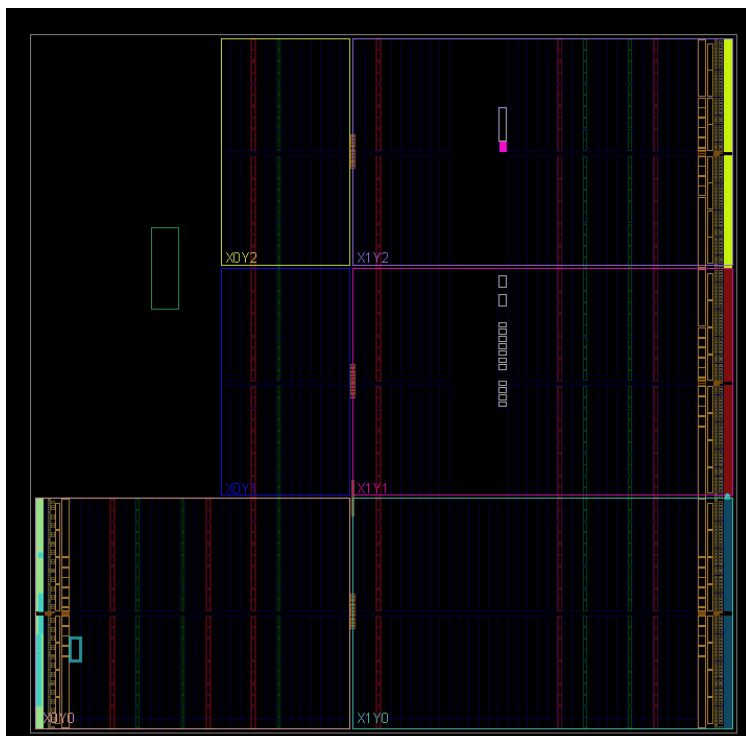
IMPLEMENTAZIONE

A seguito dell'implementazione, è possibile visualizzare le seguenti immagini, all'interno delle quali sono evidenziate in verde le porzioni utilizzate del device.

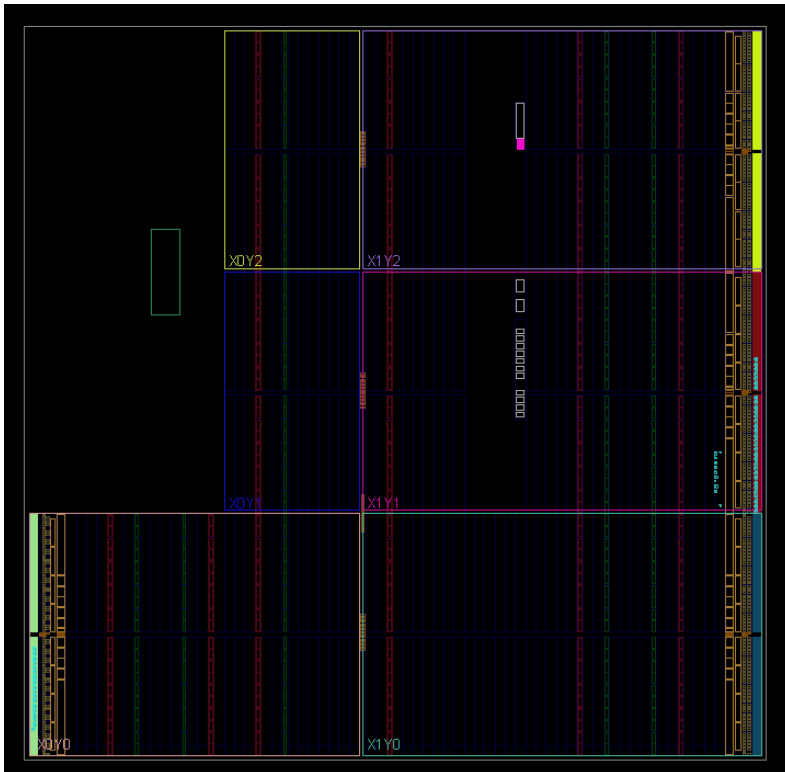
- **Caratterizzazione a 4 bit**



- **Caratterizzazione a 8 bit**



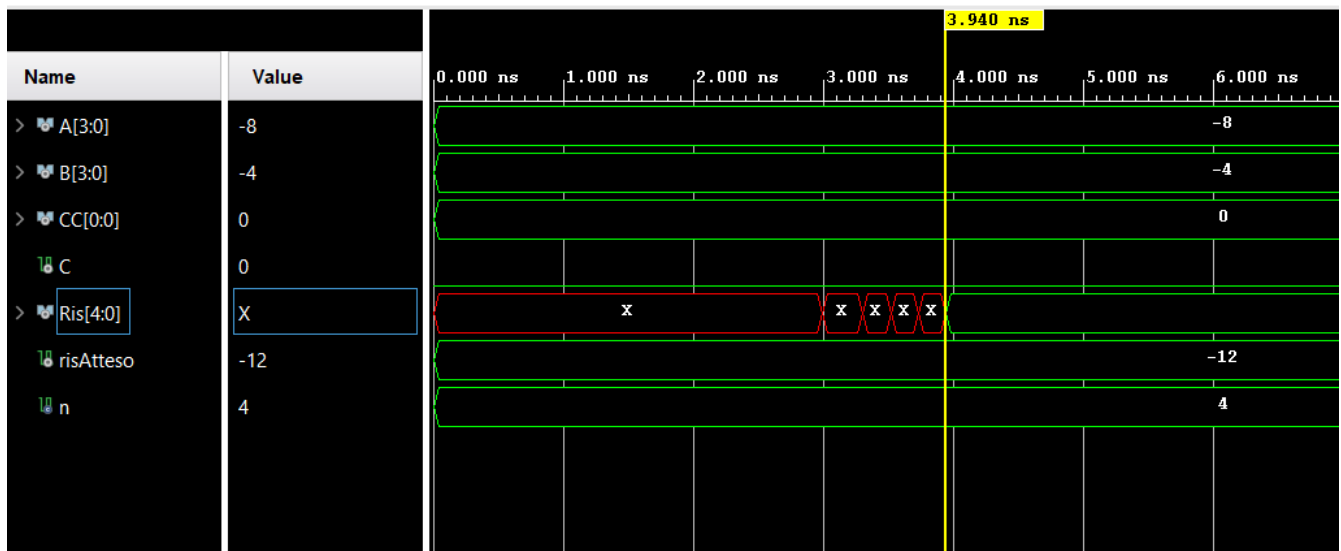
- Caratterizzazione a 16 bit



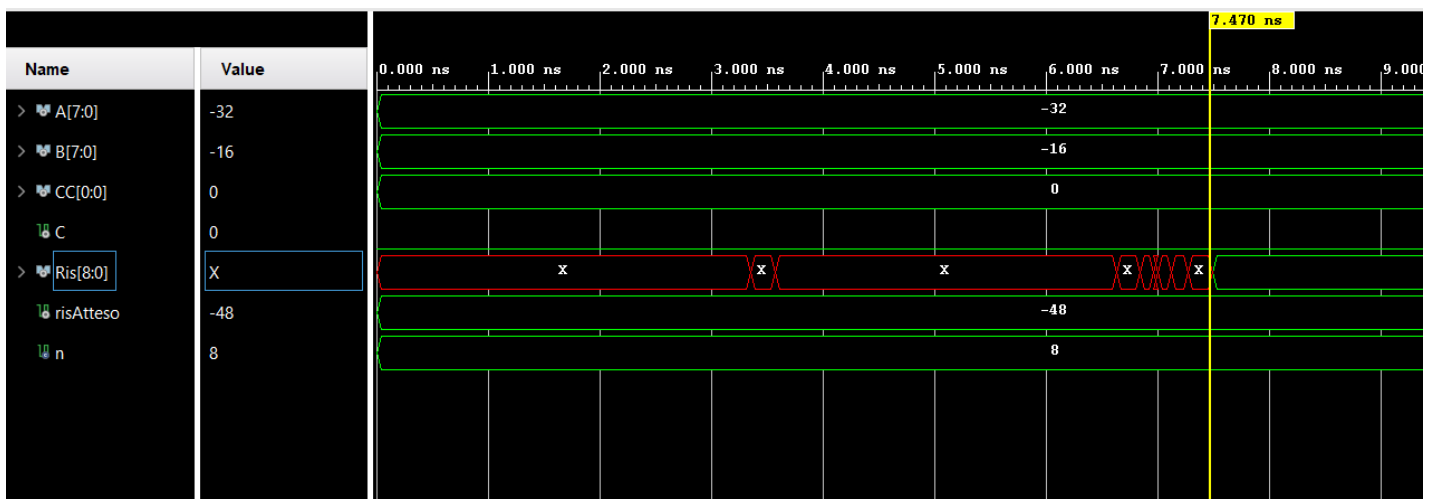
SIMULAZIONE POST-IMPLEMENTATION

Operando la simulazione post-implementation, si nota che il ritardo è maggiore rispetto a quello presente nella simulazione post-sintesi, in quanto si considera in questo caso l'implementazione del circuito su dispositivo reale.

- **Caratterizzazione a 4 bit**



- **Caratterizzazione a 8 bit**



- Caratterizzazione a 16 bit

